

7. Write a C program to simulate page replacement algorithms.

- a) FIFO
- b) LRU
- c) Optimal

CODE:

```
#include <stdio.h>
int frames[10];
void fifo(int pages[], int n, int f) {
    int i, j, k = 0, fault = 0, found;
    for(i = 0; i < f; i++)
        frames[i] = -1;
    for(i = 0; i < n; i++) {
        found = 0;
        for(j = 0; j < f; j++) {
            if(frames[j] == pages[i]) {
                found = 1;
                break;
            }
        }
        if(!found) {
            frames[k] = pages[i];
            k = (k + 1) % f;
            fault++;
        }
    }
    printf("FIFO Page Faults = %d\n", fault);
}
void lru(int pages[], int n, int f) {
    int i, j, least, pos, fault = 0;
    int time[10], counter = 0;
    for(i = 0; i < f; i++) {
        frames[i] = -1;
        time[i] = 0;
    }
    for(i = 0; i < n; i++) {
        int found = 0;
```

```

    for(j = 0; j < f; j++) {
        if(frames[j] == pages[i]) {
            counter++;
            time[j] = counter;
            found = 1;
            break;
        }
    }
    if(!found) {
        least = time[0];
        pos = 0;
        for(j = 0; j < f; j++) {
            if(frames[j] == -1) {
                pos = j;
                break;
            }
            if(time[j] < least) {
                least = time[j];
                pos = j;
            }
        }
        frames[pos] = pages[i];
        counter++;
        time[pos] = counter;
        fault++;
    }
}
printf("LRU Page Faults = %d\n", fault);
}

void optimal(int pages[], int n, int f) {
    int i, j, k, pos, farthest, fault = 0;
    for(i = 0; i < f; i++)
        frames[i] = -1;
    for(i = 0; i < n; i++) {
        int found = 0;
        for(j = 0; j < f; j++) {
            if(frames[j] == pages[i]) {

```

```

        found = 1;
        break;
    }
}
if(!found) {
    pos = -1;
    for(j = 0; j < f; j++) {
        if(frames[j] == -1) {
            pos = j;
            break;
        }
    }
    if(pos == -1) {
        farthest = i;
        pos = 0;
        for(j = 0; j < f; j++) {
            int next = -1;
            for(k = i + 1; k < n; k++) {
                if(frames[j] == pages[k]) {
                    next = k;
                    break;
                }
            }
            if(next == -1) {
                pos = j;
                break;
            }
            if(next > farthest) {
                farthest = next;
                pos = j;
            }
        }
    }
    frames[pos] = pages[i];
    fault++;
}
}

```

```

    printf("Optimal Page Faults = %d\n", fault);
}
int main() {
    int pages[20], n, f, i;
    printf("Enter number of pages: ");
    scanf("%d", &n);
    printf("Enter page reference string: ");
    for(i = 0; i < n; i++)
        scanf("%d", &pages[i]);
    printf("Enter number of frames: ");
    scanf("%d", &f);
    fifo(pages, n, f);
    lru(pages, n, f);
    optimal(pages, n, f);
    return 0;
}

```

OUTPUT:

```
Enter number of pages: 12
Enter page reference string: 7 1 0 3 0 2 5 8 9 4 6 5
Enter number of frames: 3
FIFO Page Faults = 11
LRU Page Faults = 11
Optimal Page Faults = 10

Process returned 0 (0x0)   execution time : 32.457 s
Press any key to continue.
|
```